RestSharp in the Typescript in the Playwright:

1.Post()

Method1:

```
import{test,expect} from "@playwright/test"

test("Post method with normal json request",async({request})=>{

   const requestbody={

            "firstname" : "Jim",

           "lastname" : "Brown",

           "totalprice" : 111,

            "depositpaid" : true,

             "bookingdates" : {

                  "checkin" : "2018-01-01",

                 "checkout" : "2019-01-01"

                           },

              "additionalneeds" : "Breakfast"

               }

const response=await request.post("https://restful-
booker.herokuapp.com/booking",{data:requestbody});

const responsebody=await response.json()//converting the data into the Json format

console.log("The Api data as follows:")

console.log(responsebody)

//Validate the status

expect(response.ok()).toBeTruthy()

expect(response.status()).toBe(200)

//Validate the response body

expect(responsebody).toHaveProperty("bookingid")

expect(responsebody).toHaveProperty("booking")

//Validate the booking object

const booking=responsebody.booking;
```

```
expect(booking).toMatchObject({
   //Check the output in the postman once
   //Json object validation
      "firstname": "Jim",
      "lastname": "Brown",
      "totalprice": 111,
      "depositpaid": true,
      "bookingdates": {
         "checkin": "2018-01-01",
         "checkout": "2019-01-01"
      }})
      expect(booking.bookingdates).toMatchObject({
         //Nested json object validation
         "checkin": "2018-01-01",
         "checkout": "2019-01-01"
      })


})
```

Method2:

For this type of method we need to the json file from tha file we will parse the json data and saved the file as testjsondata.json

```
{
   "firstname" : "Jim",
   "lastname" : "Brown",
   "totalprice" : 111,
   "depositpaid" : true,
   "bookingdates" : {
      "checkin" : "2018-01-01",
      "checkout" : "2019-01-01"
```

```
    },
    "additionalneeds" : "Breakfast"
}
====================================


import{test,expect} from "@playwright/test"
import fs from 'fs';
test("Post method with external json request",async({request})=>{
    const jsonpath="./tests/testdata/testjsondata.json//Identifying and saving the json file path along with the name of the file.
    let requestbody=JSON.parse(fs.readFileSync(jsonpath,'utf-8')); ";//Parsing the json file data in the json format
const response=await request.post("https://restful-booker.herokuapp.com/booking",{data:requestbody});
const responsebody=await response.json()//converting the data into the Json format
console.log("The Api data as follows:")
console.log(responsebody)
//Validate the status
expect(response.ok()).toBeTruthy()
expect(response.status()).toBe(200)
//Validate the response body
expect(responsebody).toHaveProperty("bookingid")
expect(responsebody).toHaveProperty("booking")
//Validate the booking object
const booking=responsebody.booking;
expect(booking).toMatchObject({
    //Check the output in the postman once
    //Json object validation since we are getting the data from the json file.
        "firstname": requestbody.firstname,
        "lastname": requestbody.lastname,
```

```
      "totalprice": requestbody.totalprice,
      "depositpaid": requestbody.depositpaid,
      "bookingdates":requestbody.bookingdates
   }
   )
     //Nested json object validation
      expect(booking.bookingdates).toMatchObject(requestbody.bookingdates)//Match object is used to the assertion or validation
})
```

2.Get():

Method1:Pathparameters

```
import {test,expect} from "@playwright/test"
//The data changing each time check the postman
test("Get the data through the Pathparam get()",async({request})=>{
   test.slow()
   const bookingid=1;
   //Path params for getting the data
const  response=  await request.get(`/booking/${bookingid}`);
const responsebody= await response.json();
console.log("=====Path parameters====")

console.log(responsebody);
expect (responsebody).toHaveProperty("firstname");
expect (response.status()).toBe(200);
})
```

Method2:Query parameters

```
import {test,expect} from "@playwright/test"
 test("Get the data through Query the Pathparam get()",async({request})=>{
console.log("=====Query parameters====")
```

```
//Using the Query parameters
const firstname="Eric";
const lastname="Wilson";
const Queryresponse=await request.get('/booking/',{params:{firstname,lastname}})
const Queryresponsebody=await Queryresponse.json();
console.log(Queryresponsebody)
expect (Queryresponse.status()).toBe(200);
//Output will be received in the format of the array hence assertion are done as per that
expect (Queryresponsebody.length).toBeGreaterThan(0);//Output will be fetched in the form of an array
expect ( typeof(await Queryresponsebody[0].bookingid)).toBe("number");//since its an  array
})
```

Performing Put(),Patch() and Delete():

Jsonfiles are as follows:

```
{
   "firstname" : "Jim",
   "lastname" : "Brown",
   "totalprice" : 111,
   "depositpaid" : true,
   "bookingdates" : {
      "checkin" : "2018-01-01",
      "checkout" : "2019-01-01"
   },
   "additionalneeds" : "Breakfast"
}
```

Saved as testjsondata.json

```
 {  "username" : "admin",
   "password" : "password123"}
```

Authtoken.json

==========

```
{
   "firstname" : "Bhanu Reddy",
   "lastname" : "T",
   "totalprice" : 333,
   "depositpaid" : true,
   "bookingdates" : {
      "checkin" : "2018-01-01",
      "checkout" : "2019-01-01"
   },
   "additionalneeds" : "Breakfast"
}
```

Updatejsondataput.json

==========

```
{
   "firstname" : "Nagarjuna",
   "lastname" : "B",
   "totalprice" : 8
}
```

Patchjsondata.json

==========

```
import {test,expect} from "@playwright/test"
import fs from 'fs'
/*Pre-requisites:
data:json file
create a token
```

```
1>create a booking(post)---->booking id
2>Update booking(put)//required the token
3>Delete booking(delete)//required token
*/
test("Put with the token",async({request})=>{
   //Creating the post request by using the external json file
const jsonpathforrequest="./tests/testdata/testjsondata.json";
const requestbody=JSON.parse(fs.readFileSync(jsonpathforrequest,'utf-8'));
const response=await request.post('/booking',{data:requestbody});
const responsebody=await response.json();
console.log("=====The data after posting====")
console.log(responsebody);
//Reading the data through the Get method
const bookid=1;
const getdata=await request.get(`/booking/${bookid}`);
const getdatabodyjson=await getdata.json();
console.log("======Getting data=====")
console.log(getdatabodyjson)
//Generating the token
console.log("=====Generating the Token======");
const jsonpathfortoken="./tests/testdata/Authtoken.json";
const Authtokenrequestbody=JSON.parse(fs.readFileSync(jsonpathfortoken,'utf-8'));
const Authtokenresponse=await request.post('/auth',{data:Authtokenrequestbody});
const Authtokenresponsebody=await Authtokenresponse.json();
console.log(Authtokenresponsebody);
//getting the booking id
const bookingid=responsebody.bookingid;
console.log("The booking id===>"+bookingid);
//getting the token
```

```
const token=Authtokenresponsebody.token;

console.log("The token===>"+token);

//Working with the put data

const updatedatajsonpath="./tests/testdata/updatejsondataput.json";

const updatedatarequestbody=JSON.parse(fs.readFileSync(updatedatajsonpath,'utf-8'));

//Sending the id and the token for updating the data

const updatedresponse=await
request.put(`/booking/${bookingid}`,{headers:{cookie:`token=${token}`},

                                    data:updatedatarequestbody})//Bactic operator

const updatedresponsejsonbody=await updatedresponse.json();

console.log("========The Updated data through put ======")

console.log(updatedresponsejsonbody)


//Working with the Patch data

const patchdatajsonpath="./tests/testdata/patchjsondata.json";

const patchdatarequestbody=JSON.parse(fs.readFileSync(patchdatajsonpath,'utf-8'));

//Sending the id and the token for updating the data

const patchresponse=await
request.patch(`/booking/${bookingid}`,{headers:{cookie:`token=${token}`},

                                    data:patchdatarequestbody})

const patchresponsejsonbody=await patchresponse.json();

console.log("========The Updated data througfh patch======")

console.log(patchresponsejsonbody)

//Delete operation

const daletedresponse=await
request.delete(`/booking/${bookingid}`,{headers:{cookie:`token=${token}`}});

console.log("The Id data has been deleted")

expect(daletedresponse.statusText()).toBe("Created")//StatusText() is used for  the
comparing the Text generated as response

expect(daletedresponse.status()).toBe(201)

})
```

Api Authentication methods:

1.No Authentication

2.Basic Authentication/Pre-emptive authentication

3.Bearer token

4.Api Key

```
import {test,expect} from "@playwright/test"

import { request } from "http";

/*

1>No Auth(public Api)

2>Basic Auth/Preemptive Auth(Username,Password)

3>Bearer token

4>Api Key

*/
```

1.No Authentication

```
test("No Authentication",async({request})=>{

const noauthresponse=await request.get("https://restful-
booker.herokuapp.com/booking/1");

const noauthresponsebody=await noauthresponse.json();

console.log(noauthresponsebody);

  expect(noauthresponse.ok).toBeTruthy();

    expect(noauthresponse.status()).toBe(200);

})
```

2.Basic Authentication/Pre-emptive authentication

```
test("Basic Authentication",async({request})=>{

    const basicresponse=await request.get("https://httpbin.org/basic-auth/user/pass",{

                                headers:{
```

```
                Authorization:"Basic "+Buffer.from("user:pass").toString("base64")
                //At the basic need to use the bactic operator or " "
                //After the Basic need to give 1 space else test will fail its a rule
                //In the above line Username=user and
password=pass{("user:pass")}

            }
                                    }
            );
  expect(basicresponse.ok).toBeTruthy();
  expect(basicresponse.status()).toBe(200);

})
```

3.Bearer token

```
test("Bearer Authentication",async({request})=>{
  //Create token from the Github-->account settings-->Developer settings-->personal access
tokens-->tokensclassic and generate token
  const
bearertoken="github_pat_11BN2PO7Y0WAgCabg2jLiA_VBaOfd4IrEXtzReDitmm5OKNJdDaUz
mJUOfFkQ8Apv8ROK3SBBAHa7S1Hob"
  const bearerresponse=await request.get("https://api.github.com/user/repos",{
            headers:{
                Authorization:`Bearer ${bearertoken}`
                //At the basic need to use the bactic operator only since we are
giving the token
                //After the Bearer need to give 1 space else test will fail its a rule

            }
                                    }
            );
  expect(bearerresponse.ok).toBeTruthy();
```

```
    expect(bearerresponse.status()).toBe(200);
   const  bearerresponsebody=await bearerresponse.json();
    console.log(bearerresponsebody);


})
```

4.Api Key authentication

```
/*Api key will be sent through the Query aparameters
Expect Api key remaining all the tokens will be passed through the headers
*/
```

//Not working hence its not moved ahead

```
test("Apikey Authentication",async({request})=>{
   //Create token from the Github-->account settings-->Developer settings-->personal access tokens-->tokensclassic and generate token
    const Apikeyresponse=await request.get("https://api.github.com/user/repos",{
                                params:{Apikey:"2e30ba5bb15a93fce4c8f34cf2f2736e"}


                                }


                                );
    expect(Apikeyresponse.ok).toBeTruthy();
  //  expect(Apikeyresponse.status()).toBe(200);
   const  Apikeyresponsebody=await bearerresponse.json();
    console.log(Apikeyresponsebody);


})
```

Json Schema Validator

```
import {test,expect} from "@playwright/test"
import Ajv, { JSONSchemaType } from "ajv";
```

/*install Ajv by using the cmd==>npm install ajv

==>npm install -D ajv

*Use the line==>import Ajv, { JSONSchemaType } from "ajv" ;for importing the ajv

Use the url ==>https://jsonlint.com/json-schema-generator

* under the covert options select the json schema generator

*Just copy paste the url in the piost man and copy the output data and paste it in the url

*U will get the json schema validator

*/

/*Json Schema==> first 2 lines related to schema(additional lines==>skip them) and no need to copy

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Generated Schema",
  "type": "object",
  "properties": {
    "firstName": {
      "type": "string"
    },
    "lastName": {
      "type": "string"
    },
    "city": {
      "type": "string"
    },
    "state": {
      "type": "string"
    }
  }
}*/
```

test("JsonSchema validation",async({request})=>{

```
const response=await request.get("https://mocktarget.apigee.net/json");
const responsebody=await response.json();
console.log(responsebody);
const schema={
   //Schema should have to be taken form the website of json schema validator==>https://jsonlint.com/json-schema-generator
  //For Schema generation the json values should be taken from the postman but not from the output of this test
// Json Schema==> first 2 lines related to schema(additional lines==>skip them) and no need to copy
 "type": "object",
 "properties": {
   "firstName": {
     "type": "string"
   },
   "lastName": {
     "type": "string"
   },
   "city": {
     "type": "string"
   },
   "state": {
     "type": "string"
   }
 }
}
//Validation of the schema
const ajv=new Ajv()
const v=ajv.compile(schema);//Here ajv.compile() returns v of  validatefunction() type
//We are creating the v function and passing the responsebody
```

```
//Is we are using the v at the top line means we need to use the same for the below line as well

const k=v(responsebody);//k returns true or false on comparing the schema and responsebody


  expect(k).toBeTruthy();

})

test("JsonSchema validation for normal",async({request})=>{

const response=await request.get("https://restful-booker.herokuapp.com/booking/10174");

const responsebody=await response.json();

console.log(responsebody);

const schema={

//Schema should have to be taken form the website of json schema validator==>https://jsonlint.com/json-schema-generator

   //For Schema generation the json values should be taken from the postman but not from the output of this test

// Json Schema==> first 2 lines related to schema(additional lines==>skip them) and no need to copy

  // "$schema": "https://json-schema.org/draft/2020-12/schema",

  // "title": "Generated Schema",

  "type": "object",

  "properties": {

    "firstname": {

      "type": "string"

    },

    "lastname": {

      "type": "string"

    },

    "totalprice": {

      "type": "integer"

    },
```

```
    "depositpaid": {
      "type": "boolean"
    },
    "bookingdates": {
      "type": "object",
      "properties": {
        "checkin": {
          "type": "string"
        },
        "checkout": {
          "type": "string"
        }
      }
    },
    "additionalneeds": {
      "type": "string"
    }
  }
}
//Validation of the schema
const ajv=new Ajv()
const v=ajv.compile(schema);//Here ajv.compile() returns v of  validatefunction() type
//We are creating the validate function and passing the responsebody
// we are using the v at the top line means we need to use the same for the below line as well
const k=v(responsebody);//k returns true or false on comparing the schema and responsebody
  expect(k).toBeTruthy();
})
```